Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №2
по дисциплине
«Современные системы программирования»

Выполнил: ст. гр. 320601
П. Д. Дудников
Р. В. Земляник

Проверила: Ю. О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ 3
2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 4
3 ВЫПОЛНЕНИЕ РАБОТЫ 5
3.1 Структура программы 5
3.2 Задание 1. Базовый пример с утечкой памяти 6
3.3 Задания 2 и 3. Класс мониторинга памяти и его тестирование 8
3.4 Задание 4. Статические ссылки и утечки памяти 9
3.5 Задание 5. Исследование *Large Object Heap* 11
3.6 Задание 6. Устранение упаковки 12
3.7 Задание 7. Поиск утечек в коде 13
4 КОНТРОЛЬНЫЕ ВОПРОСЫ 15
ЗАКЛЮЧЕНИЕ 17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 18
ПРИЛОЖЕНИЕ А 19

1 ЦЕЛЬ РАБОТЫ

Цель лабораторной работы – изучить механизмы сборки мусора в *.NET*, работу с поколениями памяти, выявление утечек памяти, размещение объектов в *Large Object Heap* (*LOH*) и оптимизацию за счет устранения упаковки (*boxing elimination*).

Для достижения цели в работе решены следующие задачи:

– выполнить и объяснить базовый пример с утечкой памяти из методических указаний;

– написать полный код класса мониторинга памяти *MemoryMonitor* и протестировать его;

– исследовать статическую и событийную утечки, а также перемещение объектов между поколениями;

– исследовать размещение объектов размером более 85 000 байт в *LOH* и написать метод мониторинга *LOH*;

– сравнить производительность кода с упаковкой и без нее и оптимизировать класс *UnoptimizedCode*;

– найти, объяснить и исправить утечки памяти в классе *BuggyCode*.

Все задания выполнены в одном консольном приложении *Lab2* для платформы *.NET* 10. Код примеров методических указаний перенесен без изменения логики, полный текст программы приведен в приложении А.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Память управляемых объектов в *.NET* освобождает сборщик мусора (*Garbage Collector, GC*). Объект считается мусором, если от корней приложения (статических полей, локальных переменных выполняемых методов, регистров процессора) к нему не ведет ни одна цепочка ссылок [1, 2]. Поэтому утечка памяти в управляемом коде означает не потерю указателя, а лишнюю ссылку, которая удерживает ненужный объект.

Для сокращения затрат на сборку куча разделена на поколения. Новые объекты размещаются в *Gen0*, объекты, пережившие сборку, переводятся в *Gen1*, а затем в *Gen2*, где хранятся долгоживущие данные. Сборка *Gen0* выполняется часто и быстро, сборка *Gen2* затрагивает всю кучу и обходится дорого [2]. Принудительно сборку вызывает метод *GC.Collect()*, а *GC.WaitForPendingFinalizers()* ожидает завершения финализаторов [1].

Объекты размером 85 000 байт и более размещаются в куче больших объектов *LOH*. Логически *LOH* относится к *Gen2*: такие объекты собираются только при полной сборке и по умолчанию не уплотняются, так как копирование больших блоков слишком дорого. Освободившиеся участки *LOH* образуют фрагментацию [3].

В методических указаниях выделены типичные причины утечек памяти [1]:

- неосвобожденные события и делегаты
- статические ссылки
- неправильная работа с *IDisposable*
- зацикленные ссылки

Статическое поле существует в одном экземпляре для всех объектов класса и живет до завершения приложения, поэтому все, что достижимо из статической коллекции, никогда не будет собрано. Событие хранит делегаты подписчиков, а делегат экземплярного метода содержит ссылку на объект-подписчик, поэтому издатель удерживает подписчиков до явной отписки.

Упаковка (*boxing*) – преобразование значимого типа в *object*: в куче создается объект-обертка, в который копируется значение. Распаковка (*unboxing*) – обратное извлечение значения с проверкой типа [4]. Каждая упаковка означает выделение памяти и дополнительную нагрузку на *GC*, поэтому необобщенные коллекции *ArrayList* и *List<object>* для чисел работают медленнее, чем *List<int>*.

Для анализа памяти применяются профилировщики *dotMemory* и *Visual Studio Diagnostic Tools*: сравнение снимков памяти выявляет неосвобождаемые объекты и пути к их корням *GC* [1].

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Структура программы

Программа состоит из одного файла *Program.cs*. Каждый пример методических указаний помещен в отдельное пространство имен (*Lab2.StaticReference, Lab2.Task1, Lab2.Task2* и далее), поэтому одноименные классы *MemoryMonitor* и *Program* из разных заданий не конфликтуют. Методы *Main* примеров переименованы в *Run*, а единственная точка входа *Lab2.Program.Main* выводит меню, по номеру пункта которого запускается нужное задание.

Замеры выполнены в конфигурации *Release*. В режиме *Debug JIT*-компилятор продлевает время жизни локальных переменных до конца метода, и объекты, на которые уже нет ссылок, не собираются, что искажает результаты опытов с памятью.

Перед заданиями в методических указаниях приведена демонстрация статической и экземплярной ссылок. Результат ее выполнения показан на рисунке 1.

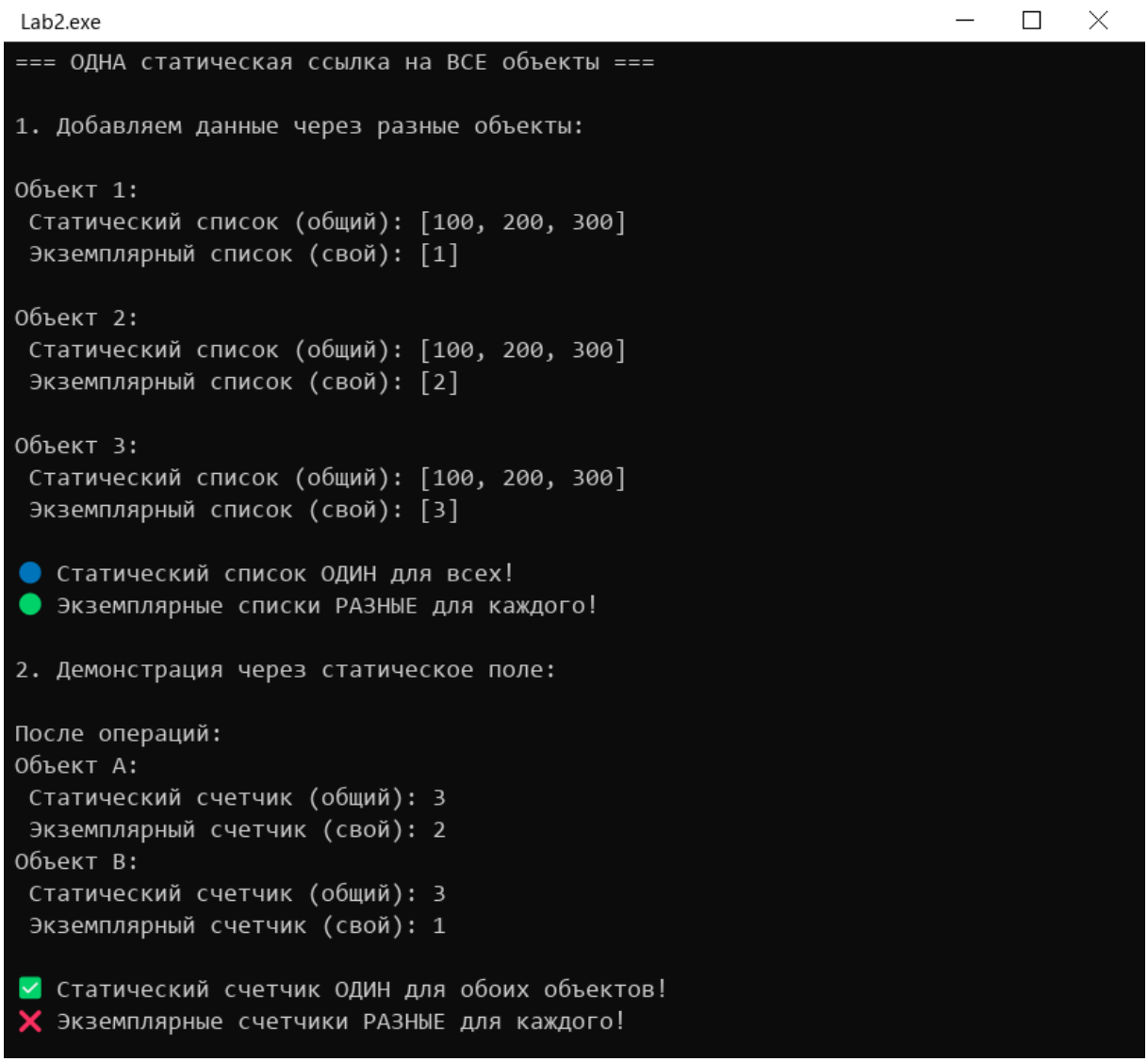


Рисунок 1 – Статическая и экземплярная ссылки

Все три объекта *StaticReferenceDemo* выводят один и тот же статический список [100, 200, 300], тогда как экземплярные списки у них разные. Статический счетчик после трех вызовов *Increment()* равен 3 для обоих объектов, а собственные счетчики равны 2 и 1. Этот пример объясняет, почему статическая коллекция становится общим корнем для данных всех объектов.

3.2 Задание 1. Базовый пример с утечкой памяти

Класс *MemoryLeakExample* добавляет в статический список 1000 массивов по 10 КБ, класс *EventLeakExample* подписывает на событие лямбда-выражения без отписки, а *MemoryMonitor* выделяет блок 10 МБ, создает 100 временных объектов *LOH* и измеряет память. Результат выполнения приведен на рисунке 2.

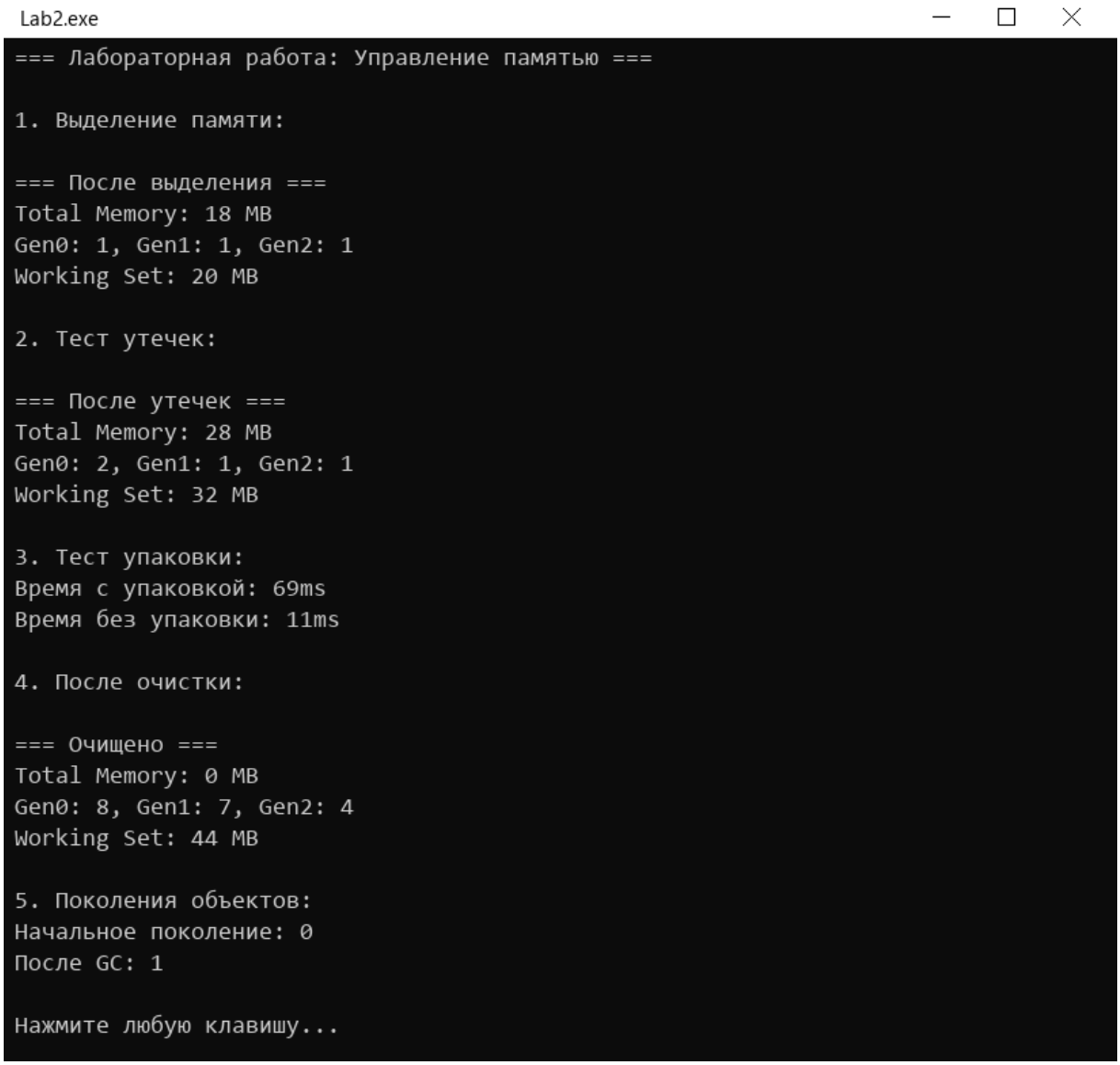


Рисунок 2 – Результат выполнения задания 1

После выделения 10 МБ и создания объектов *LOH* управляемая куча заняла 18 МБ, после создания утечек – 28 МБ: статический список удерживает около 10 МБ данных. Заполнение списка объектов миллионом чисел с упаковкой заняло 69 мс, без упаковки – 11 мс. После *ClearMemoryLeak()* и *Cleanup()* память уменьшилась до 0 МБ, так как ссылки из статического и экземплярного списков удалены. Новый объект находится в *Gen0* и после *GC.Collect()* переходит в *Gen1*.

Рабочий набор процесса (*Working Set*) после очистки вырос до 44 МБ: сборщик освободил объекты, но не сразу вернул страницы операционной системе. Поэтому для оценки утечек используется *GC.GetTotalMemory*, а не размер процесса.

3.3 Задания 2 и 3. Класс мониторинга памяти и его тестирование

По тексту методических указаний написан полный класс *MemoryMonitor*, реализующий *IDisposable*. Метод *Dispose* вызывает *Cleanup* только один раз благодаря флагу *_disposed*, а оператор *using* гарантирует вызов *Dispose* при выходе из блока. Результат теста из задания 3 показан на рисунке 3.

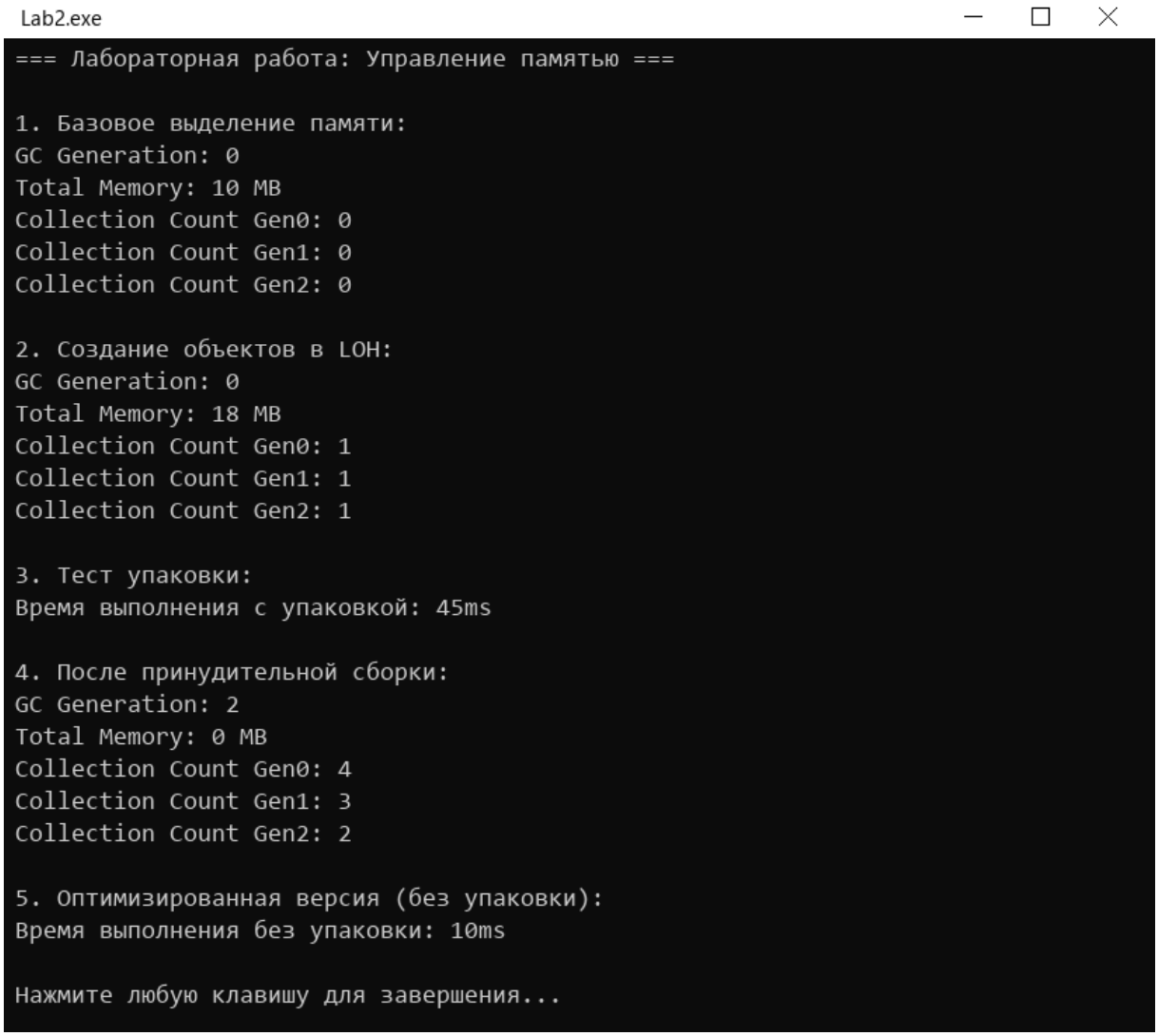


Рисунок 3 – Результат тестирования класса *MemoryMonitor*

После выделения 10 МБ показание *GC.GetTotalMemory* равно 10 МБ, после создания 100 объектов *LOH* – 18 МБ, причем выполнилась первая сборка всех поколений. Упаковка заняла 45 мс, вариант без упаковки – 10 мс. После *Cleanup()* память снизилась до 0 МБ, а объект монитора, переживший несколько сборок, перешел в *Gen2*.

3.4 Задание 4. Статические ссылки и утечки памяти

Класс *StaticLeakDemo* содержит статический список *List<byte[]>*, метод *AddData()* добавляет в него 1000 массивов по 10 КБ, а *CheckMemory()* выводит объем памяти, число объектов в списке и его поколение. Результат приведен на рисунке 4 и в таблице 1.

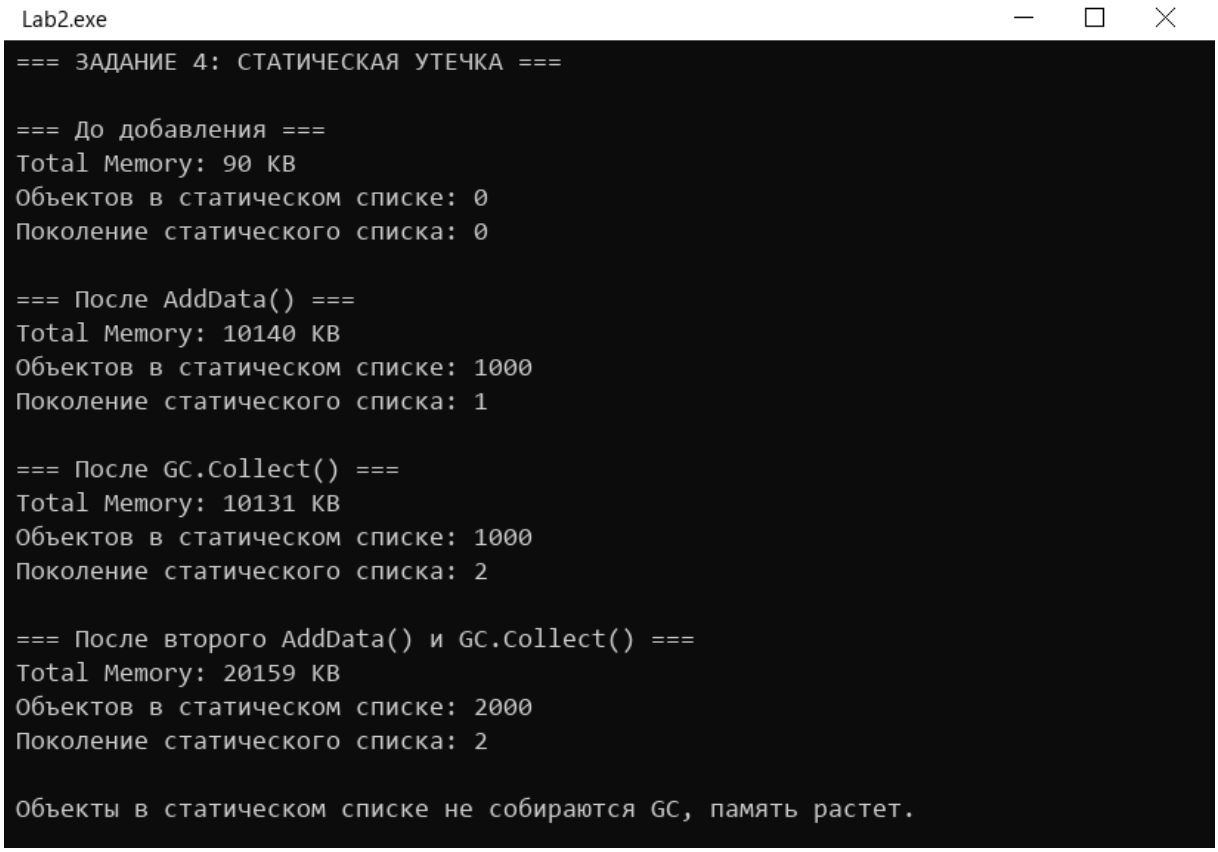


Рисунок 4 – Статическая утечка

Таблица 1 – Изменение памяти при статической утечке

Этап	Память, КБ	Объектов в списке	Поколение списка
До добавления	90	0	0
После *AddData()*	10 140	1000	1
После *GC.Collect()*	10 131	1000	2
После второго *AddData()* и *GC.Collect()*	20 159	2000	2

После *GC.Collect()* память практически не изменилась: все 1000 массивов достижимы из статического поля и сборщик их не удаляет. Повторный вызов *AddData()* удвоил объем до 20 159 КБ, то есть память растет с каждым вызовом. Сам список при сборках перешел из *Gen0* в *Gen2*, где он будет жить до завершения программы.

Класс *EventLeakDemo* демонстрирует событийную утечку. Метод *Subscribe()* создает массив 100 КБ и подписывает на событие обработчик, который захватывает этот массив и создает в теле большой объект, метод

Unsubscribe() отписывает все обработчики через *GetInvocationList()*. Результат показан на рисунке 5.

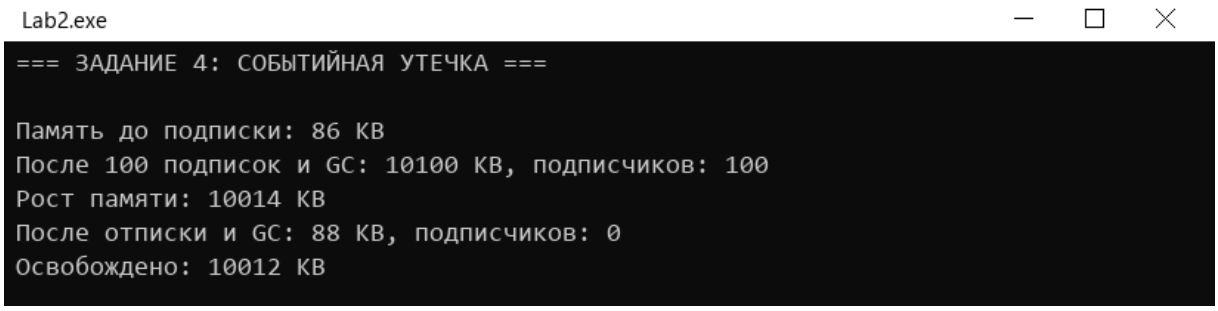


Рисунок 5 – Событийная утечка

После 100 подписок и полной сборки память выросла на 10 014 КБ – это 100 захваченных массивов по 100 КБ. Объекты не удаляются, потому что событие хранит делегаты, а делегаты через замыкание ссылаются на массивы. После отписки событие больше не ссылается на делегаты, и сборщик освободил 10 012 КБ, вернув память к исходным 88 КБ.

Для сравнения поколений создан объект, после чего трижды вызван *GC.Collect()*, а также созданы массивы разных размеров. Результат приведен на рисунке 6.

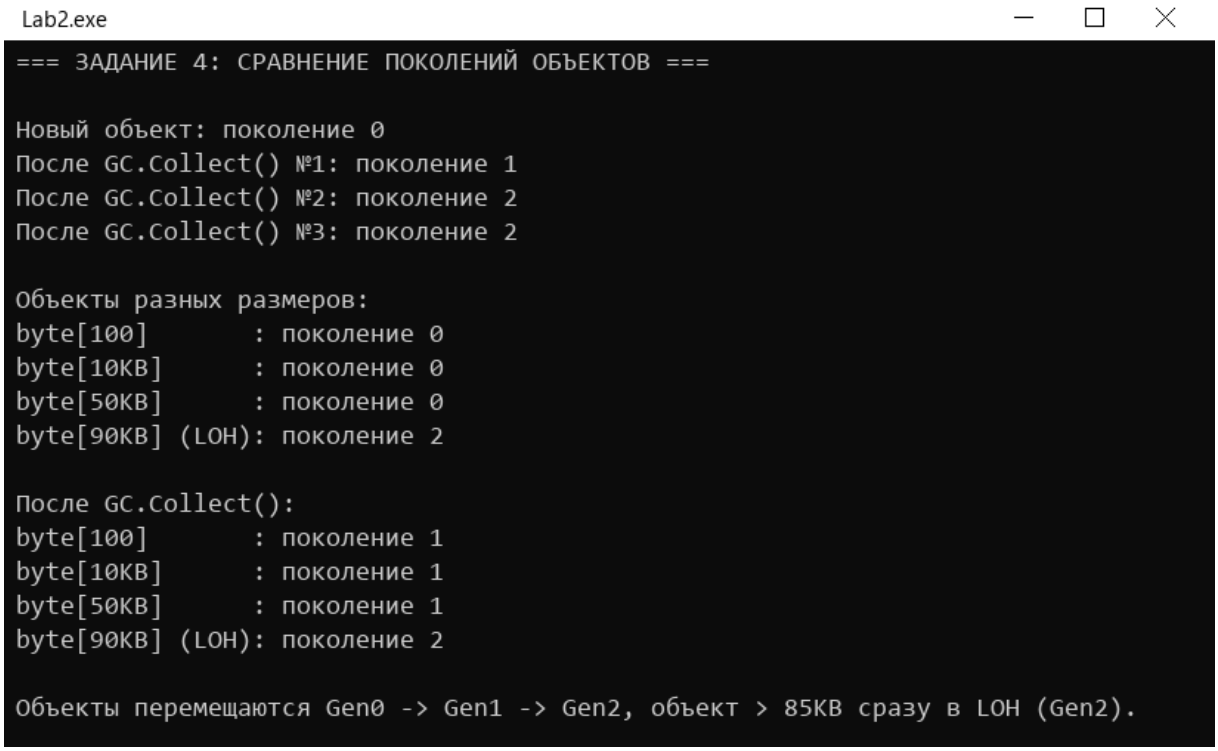


Рисунок 6 – Сравнение поколений объектов

Объект переходит из *Gen0* в *Gen1* после первой сборки и в *Gen2* после второй, дальше поколение не растет. Массивы 100 байт, 10 КБ и 50 КБ создаются в *Gen0* и после сборки переходят в *Gen1*. Массив 90 КБ (92 160 байт) сразу получает поколение 2, так как размещается в *LOH*.

3.5 Задание 5. Исследование *Large Object Heap*

Метод *AllocateLargeArrays()* выделяет 100 массивов по 100 КБ, затем выполняются замеры до и после выделения, после *GC.Collect()* и после обнуления ссылки на список. Также сравниваются массивы 90 КБ и 80 КБ. Результат показан на рисунке 7.

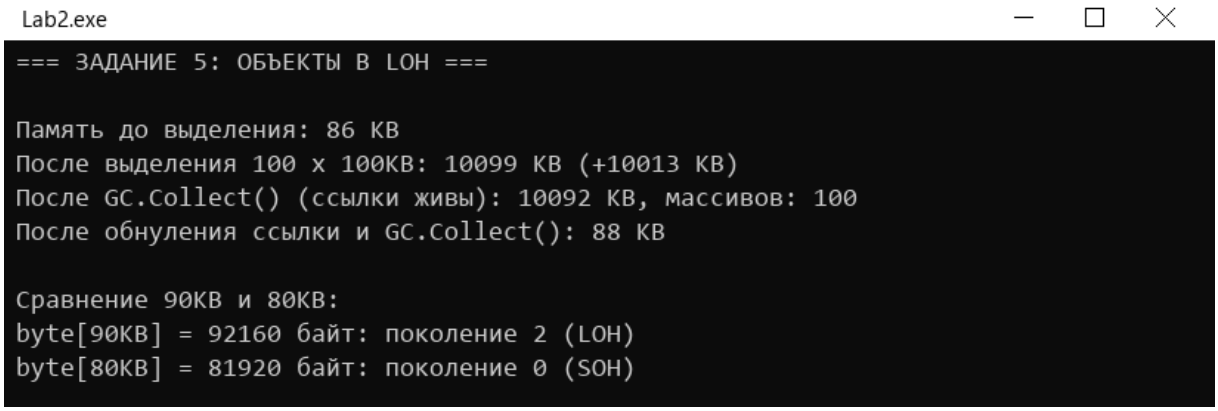


Рисунок 7 – Размещение объектов в *LOH*

Выделение увеличило память на 10 013 КБ. Пока список жив, *GC.Collect()* не освобождает массивы (10 092 КБ), после обнуления ссылки и сборки память вернулась к 88 КБ. Массив 90 КБ находится в *LOH* (поколение 2), массив 80 КБ (81 920 байт) меньше порога 85 000 байт и размещен в обычной куче (поколение 0).

Метод мониторинга *LOH* создает объекты 10, 50, 90 и 200 КБ и после каждой сборки выводит их поколение, как показано на рисунке 8 и в таблице 2.

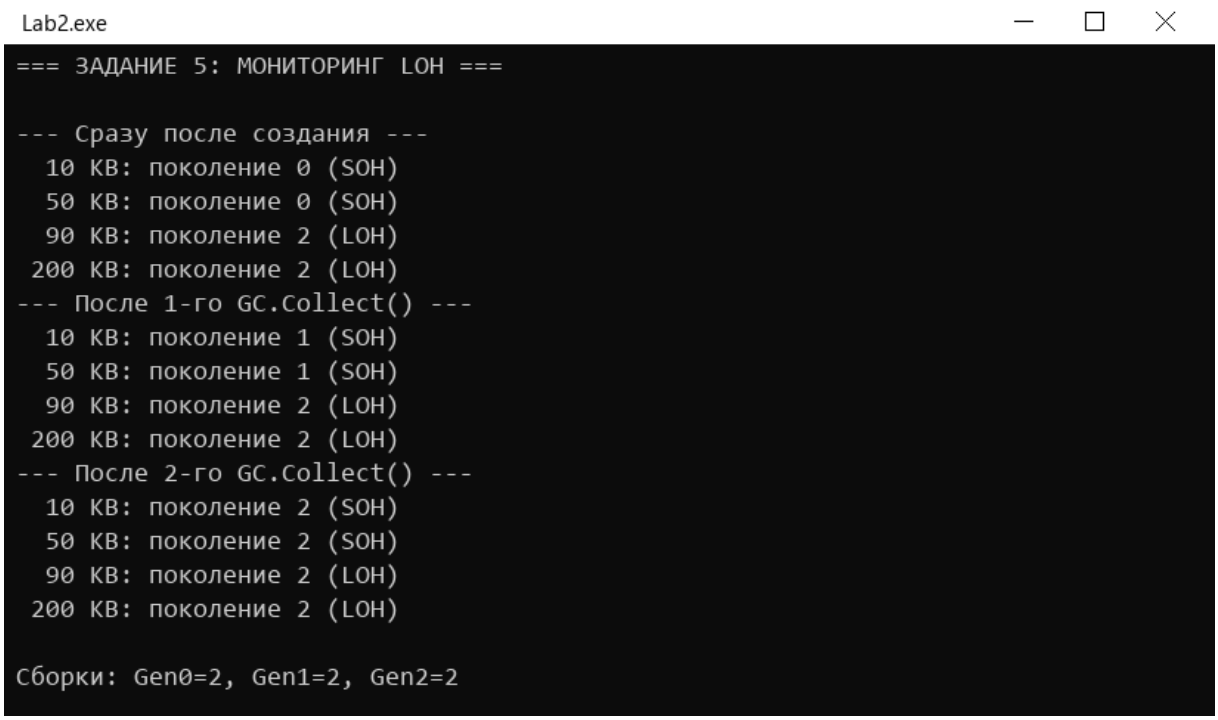


Рисунок 8 – Мониторинг *LOH*

Таблица 2 – Поколения объектов разного размера

Размер объекта	Куча	После создания	После 1-й сборки	После 2-й сборки
10 КБ	*SOH*	0	1	2
50 КБ	*SOH*	0	1	2
90 КБ	*LOH*	2	2	2
200 КБ	*LOH*	2	2	2

Объекты малой кучи (*SOH*) проходят все поколения по очереди, а большие объекты с момента создания относятся к *Gen2*. Отсюда следует, что *LOH* собирается только при полной сборке, а частое создание временных больших массивов вызывает дорогие сборки *Gen2* и фрагментацию *LOH*.

3.6 Задание 6. Устранение упаковки

Методы *ProcessWithBoxing()* и *ProcessWithoutBoxing()* добавляют 1 000 000 чисел в *List<object>* и *List<int>* и суммируют их. Перед замером выполнен прогрев *JIT*. Память измерена методом *GC.GetTotalMemory(true)* до вызова метода и после него, пока заполненный список еще жив, поэтому разность равна объему, который занимают данные. Результат приведен на рисунке 9 и в таблице 3.

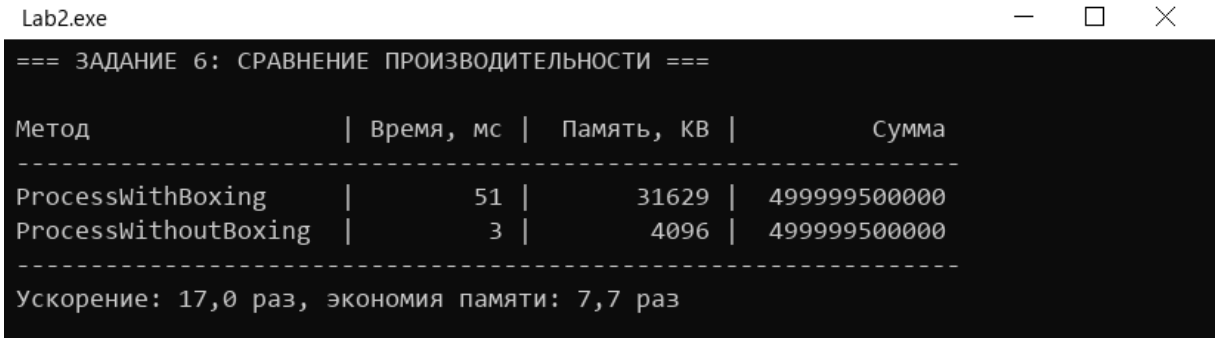


Рисунок 9 – Сравнение кода с упаковкой и без нее

Таблица 3 – Сравнение производительности при 1 000 000 операций

Метод	Время, мс	Память по *GC.GetTotalMemory*, КБ
ProcessWithBoxing (*List<object>*)	51	31 629
ProcessWithoutBoxing (*List<int>*)	3	4 096

Обе версии вернули одинаковую сумму 499 999 500 000, а вариант без упаковки занимает в 7,7 раза меньше памяти. Результат совпадает с расчетом: *List<int>* хранит значения прямо в массиве *int* (1 048 576 элементов по 4 байта, то есть 4 096 КБ), а *List<object>* хранит массив ссылок по 8 байт (8 192 КБ) и миллион отдельных объектов в куче по 24 байта (около 23 400 КБ). Время различается в 17 раз, сильнее ожидаемых в методических указаниях 2–4 раз, так как в замер входят и сборки мусора, вызванные миллионом упакованных объектов.

Класс *UnoptimizedCode* из методических указаний переписан как *OptimizedCode*: *ArrayList* заменен на *List<int>*, а цикл *foreach* (*object item*) с приведением (*int)item* – на *foreach* (*int item*). Результат показан на рисунке 10.

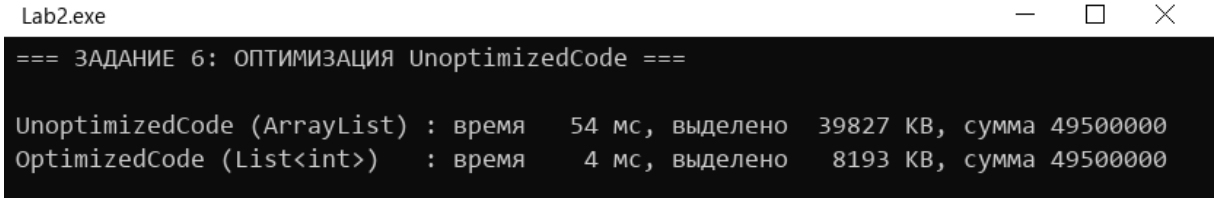


Рисунок 10 – Оптимизация класса *UnoptimizedCode*

При одинаковой сумме 49 500 000 оптимизированный класс работает за 4 мс вместо 54 мс. Здесь измерен суммарный объем выделенной памяти (*GC.GetTotalAllocatedBytes*) с учетом промежуточных массивов при росте коллекции: 8 193 КБ вместо 39 827 КБ. Ускорение объясняется отсутствием выделения объекта на каждое добавление, отсутствием проверки типа при распаковке и меньшей нагрузкой на сборщик мусора.

3.7 Задание 7. Поиск утечек в коде

В классе *BuggyCode* найдено три места утечек, их причины и исправления приведены в таблице 4.

Таблица 4 – Утечки памяти в классе *BuggyCode*

Место	Причина утечки	Исправление
static event _globalEvent и подписка в конструкторе	Статическое событие хранит делегат на каждый объект, поэтому все объекты с массивом 1 МБ живут до конца программы	Отписка *_globalEvent -= OnEvent* в *Dispose()*
static List<IDisposable> _cache	Статический список накапливает *MemoryStream* и никогда не очищается	Поток принадлежит объекту и освобождается в *Dispose()*
MemoryStream без *Dispose()*	*IDisposable*-ресурс не освобождается явно	Вызов *_stream.Dispose()* и оператор *using*
SubscribeAll()	Создает 1000 объектов, которые невозможно освободить	Каждый объект создается в блоке *using*

Исправленный класс *FixedCode* реализует *IDisposable*. В *Dispose()* он отписывается от статического события, освобождает *MemoryStream* и обнуляет ссылку на массив. Сравнение двух классов показано на рисунке 11.

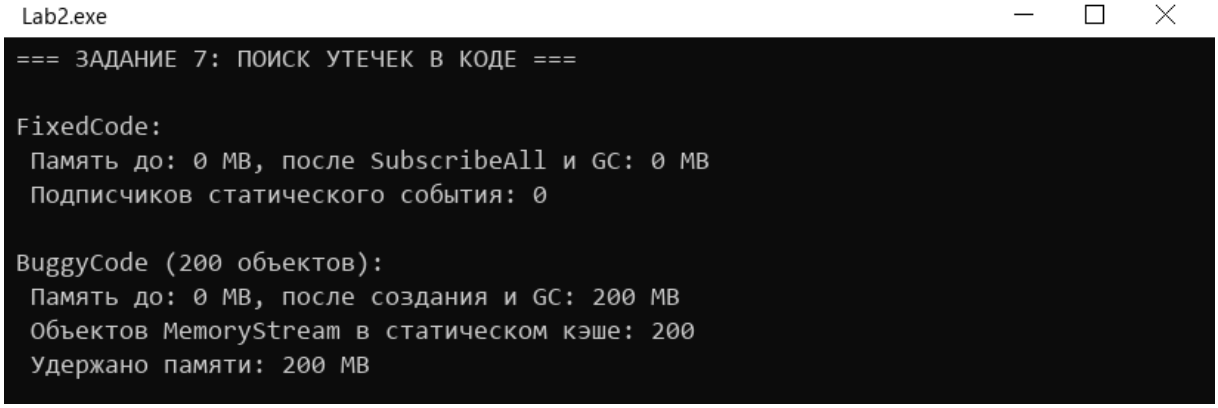


Рисунок 11 – Сравнение *BuggyCode* и *FixedCode*

После *SubscribeAll()* у *FixedCode* память осталась на уровне 0 МБ, а у события нет подписчиков. Метод *SubscribeAll()* исходного класса удерживает 1000 объектов по 1 МБ (около 1 ГБ), поэтому для замера тем же способом создано 200 объектов *BuggyCode*: после полной сборки удержано 200 МБ, а в статическом кэше осталось 200 объектов *MemoryStream*. Таким образом, утечка пропорциональна числу созданных объектов, а исправленная версия ее устраняет.

4 КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1. Почему объекты в *.NET* перемещаются между поколениями (*Gen0 → Gen1 → Gen2*) и какие последствия это имеет для производительности приложения?

Поколения основаны на наблюдении, что большинство объектов живут недолго. Объект, переживший сборку, считается долгоживущим и переводится в старшее поколение, чтобы частые дешевые сборки *Gen0* его больше не просматривали. Последствие – объекты в *Gen2* освобождаются только при дорогой полной сборке. Если долгоживущий объект, например статический кэш, хранит ссылки на временные данные, эти данные тоже попадают в *Gen2* и не освобождаются, что видно в задании 4: статический список перешел в *Gen2*, а память росла с каждым вызовом *AddData()*.

Вопрос 2. В чем принципиальная разница между статической и экземплярной ссылкой с точки зрения сборки мусора?

Экземплярная ссылка живет, пока жив объект-владелец: когда на владельца не остается ссылок, сборщик удаляет и его, и данные. Статическая ссылка является корнем *GC* и существует все время работы приложения, поэтому все, что в нее добавлено, остается достижимым. Даже небольшие объекты в статической коллекции накапливаются при каждом добавлении и никогда не удаляются без явной очистки.

Вопрос 3. Почему *LOH* не подвергается компактизации и как это влияет на производительность?

Уплотнение требует копирования живых объектов, а копирование блоков в сотни килобайт занимает много времени, поэтому по умолчанию *LOH* не уплотняется. Освобожденные участки остаются свободными промежутками. Если программа часто создает большие массивы разного размера, новый массив может не поместиться ни в один промежуток, и куча растет при большом объеме свободной памяти. Кроме того, каждая сборка *LOH* является полной сборкой *Gen2*. Для снижения проблемы используют повторное использование буферов (*ArrayPool*) или разовое уплотнение через *GCSettings.LargeObjectHeapCompactionMode*.

Вопрос 4. Почему неподписанные события являются частой причиной утечек памяти?

При подписке издатель сохраняет делегат, который содержит ссылку на объект-подписчик. Пока жив издатель, жив и подписчик, даже если в программе на него больше нет ссылок. Особенно опасны статические события и долгоживущие издатели. Исправление – отписываться оператором -= в *Dispose()* или по завершении работы, как в классе *FixedCode*. Безопасная

подписка выглядит так: объект подписывается в конструкторе, реализует *IDisposable* и в *Dispose()* выполняет отписку, а создается в блоке *using*.

Вопрос 5. Что такое упаковка и почему она негативно влияет на производительность и память?

Упаковка – копирование значимого типа в новый объект в куче при приведении к *object*. На каждый элемент выделяется отдельный объект с заголовком, коллекция хранит ссылки, а при чтении выполняется распаковка с проверкой типа. Это увеличивает объем памяти и число сборок мусора. Оптимизация состоит в замене *ArrayList* и *List<object>* обобщенной коллекцией *List<int>*, которая хранит значения без упаковки. В задании 6 это уменьшило объем памяти под данные в 7,7 раза и ускорило обработку в 13–17 раз.

ЗАКЛЮЧЕНИЕ

В лабораторной работе изучены сборка мусора в *.NET*, поколения объектов, куча больших объектов и типичные причины утечек памяти. Все задания выполнены в одном консольном приложении, код методических указаний использован без изменения логики.

Опыты подтвердили теоретические положения. Массивы в статическом списке не освобождаются после *GC.Collect()*, и память растет с каждым вызовом *AddData()*. Событие с 100 подписками удерживало около 10 МБ, которые освободились после отписки. Объекты малой кучи последовательно переходят из *Gen0* в *Gen2*, а массивы больше 85 000 байт сразу относятся к *Gen2*.

Устранение упаковки уменьшило объем памяти под данные в 7,7 раза и ускорило обработку миллиона чисел. В классе *BuggyCode* найдены утечки через статическое событие, статический кэш и неосвобожденный *MemoryStream*, исправленный класс *FixedCode* с *IDisposable* не оставляет удерживаемой памяти. Ограничение результатов – время и объемы памяти зависят от компьютера и состояния процесса, поэтому важны соотношения величин, а не абсолютные значения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Управление памятью в *C#* и анализ утечек : методические указания к лабораторной работе № 2 по дисциплине «Современные системы программирования». – Минск : БГУИР, 2026.

2 Основы сборки мусора [Электронный ресурс] // *Microsoft Learn*. – Режим доступа: *https*://*learn.microsoft.com/ru-ru/dotnet/standard/garbage-collection/fundamentals*. – Дата доступа: 22.10.2026.

3 Куча больших объектов в системах *Windows* [Электронный ресурс] // *Microsoft Learn*. – Режим доступа: *https*://*learn.microsoft.com/ru-ru/dotnet/standard/garbage-collection/large-object-heap*. – Дата доступа: 22.10.2026.

4 Упаковка-преобразование и распаковка-преобразование [Электронный ресурс] // *Microsoft Learn*. – Режим доступа: *https*://*learn.microsoft.com/ru-ru/dotnet/csharp/programming-guide/types/boxing-and-unboxing*. – Дата доступа: 22.10.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программы Program.cs

```
// Лабораторная работа 2. Управление памятью в C# и анализ утечек
// Все задания собраны в одном файле. Каждый пример методички лежит в своем
// пространстве имен, а его метод Main переименован в Run и вызывается из
меню.

using System;
using System.Collections;
using System.Collections.Generic;
using System.Diagnostics;
using System.IO;
using System.Text;

namespace Lab2
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;

            while (true)
            {
                Console.WriteLine("\n=== Лабораторная работа 2: Управление
памятью и анализ утечек ===");
                Console.WriteLine("1 - Статическая и экземплярная ссылки
(демонстрация)");
                Console.WriteLine("2 - Задание 1. Базовый пример с утечкой
памяти");
                Console.WriteLine("3 - Задания 2 и 3. MemoryMonitor :
IDisposable и тестирование");
                Console.WriteLine("4 - Задание 4. Статическая утечка");
                Console.WriteLine("5 - Задание 4. Событийная утечка");
                Console.WriteLine("6 - Задание 4. Сравнение поколений
объектов");
                Console.WriteLine("7 - Задание 5. Объекты в LOH");
                Console.WriteLine("8 - Задание 5. Мониторинг LOH");
                Console.WriteLine("9 - Задание 6. Сравнение
производительности boxing");
                Console.WriteLine("10 - Задание 6. Оптимизация
UnoptimizedCode");
                Console.WriteLine("11 - Задание 7. BuggyCode и исправленная
версия");
                Console.WriteLine("0 - Выход");
                Console.Write("> ");

                string choice = Console.ReadLine();
                if (choice == null || choice == "0") break;

                switch (choice)
                {
                    case "1": StaticReference.Program.Run(); break;
                    case "2": Task1.Program.Run(); break;
                    case "3": Task2.Program.Run(); break;
                    case "4": Task4.StaticLeakDemo.Run(); break;
                    case "5": Task4.EventLeakDemo.Run(); break;
                    case "6": Task4.GenerationsDemo.Run(); break;
```

```
                    case "7": Task5.LOHResearch.Run(); break;
                    case "8": Task5.LOHMonitor.Run(); break;
                    case "9": Task6.BoxingComparison.Run(); break;
                    case "10": Task6.OptimizationDemo.Run(); break;
                    case "11": Task7.LeakSearch.Run(); break;
                    default: Console.WriteLine("Неизвестный пункт меню");
break;
                }
            }
        }
    }
}

// ====================== Статическая ссылка (пример из методички)
====================
namespace Lab2.StaticReference
{
    public class StaticReferenceDemo
    {
        // СТАТИЧЕСКАЯ ССЫЛКА - ОДНА для ВСЕХ объектов
        private static List<int> _staticList = new List<int>();

        // ЭКЗЕМПЛЯРНАЯ ССЫЛКА - своя для КАЖДОГО объекта
        private List<int> _instanceList = new List<int>();

        public void AddToStatic(int value)
        {
            _staticList.Add(value); // Все объекты используют ОДИН и тот же
список
        }

        public void AddToInstance(int value)
        {
            _instanceList.Add(value); // Каждый объект использует СВОЙ список
        }

        public void PrintLists(string objectName)
        {
            Console.WriteLine($"{objectName}:");
            Console.WriteLine($" Статический список (общий): [{string.Join(",
", _staticList)}]");
            Console.WriteLine($" Экземплярный список (свой): [{string.Join(",
", _instanceList)}]");
            Console.WriteLine();
        }
    }

    class Program
    {
        public static void Run()
        {
            Console.WriteLine("=== ОДНА статическая ссылка на ВСЕ объекты
===\n");

            // Создаем три разных объекта
            var obj1 = new StaticReferenceDemo();
            var obj2 = new StaticReferenceDemo();
            var obj3 = new StaticReferenceDemo();

            // Добавляем данные через разные объекты
            Console.WriteLine("1. Добавляем данные через разные объекты:\n");
```

```
            obj1.AddToStatic(100);   // Статический список для ВСЕХ объектов
            obj1.AddToInstance(1);   // Только для obj1

            obj2.AddToStatic(200);   // Добавляем в ТОТ ЖЕ статический список
            obj2.AddToInstance(2);   // Только для obj2

            obj3.AddToStatic(300);   // Опять в ТОТ ЖЕ статический список
            obj3.AddToInstance(3);   // Только для obj3

            // Выводим состояние каждого объекта
            obj1.PrintLists("Объект 1");
            obj2.PrintLists("Объект 2");
            obj3.PrintLists("Объект 3");

            Console.WriteLine("🔘 Статический список ОДИН для всех!");
            Console.WriteLine("🔘 Экземплярные списки РАЗНЫЕ для
каждого!\n");

            // Демонстрация через статическое поле
            Console.WriteLine("2. Демонстрация через статическое поле:\n");
            StaticFieldDemo.ShowStaticFieldDemo();
        }
    }

    public class StaticFieldDemo
    {
        private static int _staticCounter = 0;   // ОДИН счетчик для всех
        private int _instanceCounter = 0;        // СВОЙ счетчик для каждого

        public void Increment()
        {
            _staticCounter++;    // Увеличивает ОБЩИЙ счетчик
            _instanceCounter++;  // Увеличивает СВОЙ счетчик
        }

        public void ShowCounters(string objectName)
        {
            Console.WriteLine($"{objectName}:");
            Console.WriteLine($" Статический счетчик (общий):
{_staticCounter}");
            Console.WriteLine($" Экземплярный счетчик (свой):
{_instanceCounter}");
        }

        public static void ShowStaticFieldDemo()
        {
            var objA = new StaticFieldDemo();
            var objB = new StaticFieldDemo();

            objA.Increment(); // staticCounter = 1, instanceCounter objA = 1
            objA.Increment(); // staticCounter = 2, instanceCounter objA = 2
            objB.Increment(); // staticCounter = 3, instanceCounter objB = 1

            Console.WriteLine("После операций:");
            objA.ShowCounters("Объект A");
            objB.ShowCounters("Объект B");

            Console.WriteLine("\n☑ Статический счетчик ОДИН для обоих
объектов!");
            Console.WriteLine("❌ Экземплярные счетчики РАЗНЫЕ для
каждого!");
```

```
        }
    }
}

// ====================== Задание 1. Базовый пример с утечкой памяти
====================
namespace Lab2.Task1
{
    public class MemoryLeakExample
    {
        private static List<byte[]> _staticList = new List<byte[]>();

        public static void CreateMemoryLeak()
        {
            // Создаем объекты, которые никогда не будут собраны
            for (int i = 0; i < 1000; i++)
            {
                _staticList.Add(new byte[1024 * 10]); // 10KB каждый
            }
        }

        public static void ClearMemoryLeak()
        {
            _staticList.Clear();
            GC.Collect();
            GC.WaitForPendingFinalizers();
        }
    }

    public class EventLeakExample
    {
        public event EventHandler BigEvent;

        public void SubscribeLeak()
        {
            // Подписка на событие без отписки
            BigEvent += (sender, e) =>
            {
                var data = new byte[1024 * 100];
            };
        }
    }

    public class MemoryMonitor
    {
        private List<byte[]> _allocatedMemory = new List<byte[]>();

        public void AllocateMemory(int sizeInMB)
        {
            _allocatedMemory.Add(new byte[sizeInMB * 1024 * 1024]);
        }

        public void AllocateLOHObjects(int count)
        {
            for (int i = 0; i < count; i++)
            {
                var largeObject = new byte[90000 + i];
            }
        }

        public void SimulateBoxing(int count = 1000000)
        {
```

```
            var list = new List<object>();
            for (int i = 0; i < count; i++)
            {
                list.Add(i);
            }
        }

        public void PrintMemoryInfo(string message = "")
        {
            if (!string.IsNullOrEmpty(message))
                Console.WriteLine($"\n=== {message} ===");

            Console.WriteLine($"Total Memory: {GC.GetTotalMemory(false) /
(1024 * 1024)} MB");
            Console.WriteLine($"Gen0: {GC.CollectionCount(0)}, Gen1:
{GC.CollectionCount(1)}, Gen2: {GC.CollectionCount(2)}");

            using (Process process = Process.GetCurrentProcess())
            {
                Console.WriteLine($"Working Set: {process.WorkingSet64 /
(1024 * 1024)} MB");
            }
        }

        public void Cleanup()
        {
            _allocatedMemory.Clear();
            GC.Collect();
            GC.WaitForPendingFinalizers();
        }
    }

    // Large Object Heap - пример
    public class LOHExample
    {
        public static void CreateLargeObjects()
        {
            // Объекты > 85KB попадают в LOH
            for (int i = 0; i < 100; i++)
            {
                var largeArray = new byte[100000]; // ~100KB
                // Используем largeArray
            }
        }
    }

    // Boxing и его влияние
    public class BoxingExample
    {
        private List<object> _boxedItems = new List<object>();
        private List<int> _unboxedItems = new List<int>();

        public void BoxingOperation()
        {
            // Упаковка - создает дополнительные объекты в куче
            for (int i = 0; i < 1000000; i++)
            {
                _boxedItems.Add(i); // int упаковывается в object
            }
        }

        public void UnboxedOperation()
```

```
        {
            // Без упаковки - эффективнее
            for (int i = 0; i < 1000000; i++)
            {
                _unboxedItems.Add(i);
            }
        }
    }

    class Program
    {
        public static void Run()
        {
            Console.WriteLine("=== Лабораторная работа: Управление памятью
===");

            var monitor = new MemoryMonitor();

            // 1. Тест памяти и LOH
            Console.WriteLine("\n1. Выделение памяти:");
            monitor.AllocateMemory(10);
            monitor.AllocateLOHObjects(100);
            monitor.PrintMemoryInfo("После выделения");

            // 2. Тест утечек
            Console.WriteLine("\n2. Тест утечек:");
            MemoryLeakExample.CreateMemoryLeak();

            var eventExample = new EventLeakExample();
            for (int i = 0; i < 100; i++)
                eventExample.SubscribeLeak();

            monitor.PrintMemoryInfo("После утечек");

            // 3. Тест Boxing
            Console.WriteLine("\n3. Тест упаковки:");
            var sw = Stopwatch.StartNew();
            monitor.SimulateBoxing(1000000);
            sw.Stop();
            Console.WriteLine($"Время с упаковкой:
{sw.ElapsedMilliseconds}ms");

            // Без упаковки
            sw.Restart();
            var optimizedList = new List<int>();
            for (int i = 0; i < 1000000; i++)
                optimizedList.Add(i);
            sw.Stop();
            Console.WriteLine($"Время без упаковки:
{sw.ElapsedMilliseconds}ms");

            // 4. Очистка
            Console.WriteLine("\n4. После очистки:");
            MemoryLeakExample.ClearMemoryLeak();
            monitor.Cleanup();
            monitor.PrintMemoryInfo("Очищено");

            // 5. Поколения объектов
            Console.WriteLine("\n5. Поколения объектов:");
            var obj = new object();
            Console.WriteLine($"Начальное поколение:
{GC.GetGeneration(obj)}");
```

```
            GC.Collect();
            Console.WriteLine($"После GC: {GC.GetGeneration(obj)}");

            Console.WriteLine("\nНажмите любую клавишу...");
            Console.ReadKey();
        }
    }
}

// ====================== Задания 2 и 3. Класс мониторинга памяти и
тестирование ======================
namespace Lab2.Task2
{
    public class MemoryMonitor : IDisposable
    {
        private bool _disposed;
        private List<byte[]> _allocatedMemory = new List<byte[]>();
        private Random _random = new Random();

        public void AllocateMemory(int sizeInMB)
        {
            // Выделяем память с возможной утечкой
            var data = new byte[sizeInMB * 1024 * 1024];
            _allocatedMemory.Add(data);
        }

        public void AllocateLOHObjects(int count)
        {
            for (int i = 0; i < count; i++)
            {
                // Объекты > 85KB
                var largeObject = new byte[90000 + i];
            }
        }

        public void SimulateBoxing()
        {
            var list = new List<object>();
            for (int i = 0; i < 1000000; i++)
            {
                list.Add(i); // Упаковка
            }
        }

        public void PrintMemoryInfo()
        {
            Console.WriteLine($"GC Generation: {GC.GetGeneration(this)}");
            Console.WriteLine($"Total Memory: {GC.GetTotalMemory(false) /
(1024 * 1024)} MB");
            Console.WriteLine($"Collection Count Gen0:
{GC.CollectionCount(0)}");
            Console.WriteLine($"Collection Count Gen1:
{GC.CollectionCount(1)}");
            Console.WriteLine($"Collection Count Gen2:
{GC.CollectionCount(2)}");
        }

        public void Cleanup()
        {
            _allocatedMemory.Clear();
            GC.Collect();
            GC.WaitForPendingFinalizers();
```

```
        }

        public void Dispose()
        {
            if (!_disposed)
            {
                Cleanup();
                _disposed = true;
            }
        }
    }

    class Program
    {
        public static void Run()
        {
            Console.WriteLine("=== Лабораторная работа: Управление памятью
===");

            using (var monitor = new MemoryMonitor())
            {
                // 1. Тестирование базовой памяти
                Console.WriteLine("\n1. Базовое выделение памяти:");
                monitor.AllocateMemory(10);
                monitor.PrintMemoryInfo();

                // 2. Тестирование LOH
                Console.WriteLine("\n2. Создание объектов в LOH:");
                monitor.AllocateLOHObjects(100);
                monitor.PrintMemoryInfo();

                // 3. Тестирование Boxing
                Console.WriteLine("\n3. Тест упаковки:");
                var stopwatch = Stopwatch.StartNew();
                monitor.SimulateBoxing();
                stopwatch.Stop();
                Console.WriteLine($"Время выполнения с упаковкой:
{stopwatch.ElapsedMilliseconds}ms");

                // 4. Принудительная сборка
                Console.WriteLine("\n4. После принудительной сборки:");
                monitor.Cleanup();
                monitor.PrintMemoryInfo();

                // 5. Оптимизированная версия
                Console.WriteLine("\n5. Оптимизированная версия (без
упаковки):");
                stopwatch.Restart();
                var optimizedList = new List<int>();
                for (int i = 0; i < 1000000; i++)
                {
                    optimizedList.Add(i);
                }
                stopwatch.Stop();
                Console.WriteLine($"Время выполнения без упаковки:
{stopwatch.ElapsedMilliseconds}ms");
            }

            Console.WriteLine("\nНажмите любую клавишу для завершения...");
            Console.ReadKey();
        }
    }
```

```
}

// ====================== Задание 4. Статические ссылки, события, поколения
======================
namespace Lab2.Task4
{
    // Статическая утечка: список живет столько же, сколько приложение
    public class StaticLeakDemo
    {
        private static List<byte[]> _staticList = new List<byte[]>();

        public static void AddData()
        {
            for (int i = 0; i < 1000; i++)
            {
                _staticList.Add(new byte[1024 * 10]); // 10KB каждый
            }
        }

        public static void CheckMemory(string message)
        {
            Console.WriteLine($"\n=== {message} ===");
            Console.WriteLine($"Total Memory: {GC.GetTotalMemory(false) /
1024} KB");
            Console.WriteLine($"Объектов в статическом списке:
{_staticList.Count}");
            Console.WriteLine($"Поколение статического списка:
{GC.GetGeneration(_staticList)}");
        }

        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 4: СТАТИЧЕСКАЯ УТЕЧКА ===");

            CheckMemory("До добавления");

            AddData();
            CheckMemory("После AddData()");

            GC.Collect();
            GC.WaitForPendingFinalizers();
            CheckMemory("После GC.Collect()");

            AddData();
            GC.Collect();
            CheckMemory("После второго AddData() и GC.Collect()");

            Console.WriteLine("\nОбъекты в статическом списке не собираются
GC, память растет.");
        }
    }

    // Событийная утечка: издатель хранит ссылки на обработчики
    public class EventLeakDemo
    {
        public event EventHandler BigEvent;

        public void Subscribe()
        {
            var data = new byte[1024 * 100]; // 100KB, захватывается
обработчиком
```

```
            BigEvent += (sender, e) =>
            {
                var temp = new byte[data.Length]; // большой объект в
обработчике
            };
        }

        public void Unsubscribe()
        {
            if (BigEvent == null) return;

            foreach (EventHandler handler in BigEvent.GetInvocationList())
            {
                BigEvent -= handler;
            }
        }

        public int SubscribersCount => BigEvent?.GetInvocationList().Length
?? 0;

        public void RaiseEvent()
        {
            BigEvent?.Invoke(this, EventArgs.Empty);
        }

        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 4: СОБЫТИЙНАЯ УТЕЧКА ===");

            var demo = new EventLeakDemo();
            long before = GC.GetTotalMemory(true);
            Console.WriteLine($"\nПамять до подписки: {before / 1024} KB");

            for (int i = 0; i < 100; i++)
            {
                demo.Subscribe();
            }
            demo.RaiseEvent();

            long afterSubscribe = GC.GetTotalMemory(true);
            Console.WriteLine($"После 100 подписок и GC: {afterSubscribe /
1024} KB, подписчиков: {demo.SubscribersCount}");
            Console.WriteLine($"Рост памяти: {(afterSubscribe - before) /
1024} KB");

            demo.Unsubscribe();

            long afterUnsubscribe = GC.GetTotalMemory(true);
            Console.WriteLine($"После отписки и GC: {afterUnsubscribe / 1024}
KB, подписчиков: {demo.SubscribersCount}");
            Console.WriteLine($"Освобождено: {(afterSubscribe -
afterUnsubscribe) / 1024} KB");
        }
    }

    // Перемещение объектов между поколениями
    public class GenerationsDemo
    {
        public static void Run()
        {
```

```
            Console.WriteLine("=== ЗАДАНИЕ 4: СРАВНЕНИЕ ПОКОЛЕНИЙ ОБЪЕКТОВ
===\n");

            var obj = new object();
            Console.WriteLine($"Новый объект: поколение
{GC.GetGeneration(obj)}");

            for (int i = 1; i <= 3; i++)
            {
                GC.Collect();
                Console.WriteLine($"После GC.Collect() №{i}: поколение
{GC.GetGeneration(obj)}");
            }

            Console.WriteLine("\nОбъекты разных размеров:");
            var small = new byte[100];
            var medium = new byte[1024 * 10];
            var big = new byte[1024 * 50];
            var large = new byte[1024 * 90]; // > 85KB

            Console.WriteLine($"byte[100]       : поколение
{GC.GetGeneration(small)}");
            Console.WriteLine($"byte[10KB]      : поколение
{GC.GetGeneration(medium)}");
            Console.WriteLine($"byte[50KB]      : поколение
{GC.GetGeneration(big)}");
            Console.WriteLine($"byte[90KB] (LOH): поколение
{GC.GetGeneration(large)}");

            GC.Collect();
            Console.WriteLine("\nПосле GC.Collect():");
            Console.WriteLine($"byte[100]       : поколение
{GC.GetGeneration(small)}");
            Console.WriteLine($"byte[10KB]      : поколение
{GC.GetGeneration(medium)}");
            Console.WriteLine($"byte[50KB]      : поколение
{GC.GetGeneration(big)}");
            Console.WriteLine($"byte[90KB] (LOH): поколение
{GC.GetGeneration(large)}");

            Console.WriteLine("\nОбъекты перемещаются Gen0 -> Gen1 -> Gen2,
объект > 85KB сразу в LOH (Gen2).");
        }
    }
}

// ====================== Задание 5. Исследование Large Object Heap
====================
namespace Lab2.Task5
{
    public class LOHResearch
    {
        // Выделяет 100 массивов по 100KB (объекты > 85KB попадают в LOH)
        public static List<byte[]> AllocateLargeArrays()
        {
            var arrays = new List<byte[]>();
            for (int i = 0; i < 100; i++)
            {
                arrays.Add(new byte[100 * 1024]);
            }
            return arrays;
        }
```

```
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 5: ОБЪЕКТЫ В LOH ===\n");

            long before = GC.GetTotalMemory(true);
            Console.WriteLine($"Память до выделения: {before / 1024} KB");

            var arrays = AllocateLargeArrays();
            long after = GC.GetTotalMemory(false);
            Console.WriteLine($"После выделения 100 x 100KB: {after / 1024}
KB (+{(after - before) / 1024} KB)");

            GC.Collect();
            long afterCollect = GC.GetTotalMemory(false);
            Console.WriteLine($"После GC.Collect() (ссылки живы):
{afterCollect / 1024} KB, массивов: {arrays.Count}");

            arrays = null;
            GC.Collect();
            long afterRelease = GC.GetTotalMemory(false);
            Console.WriteLine($"После обнуления ссылки и GC.Collect():
{afterRelease / 1024} KB");

            Console.WriteLine("\nСравнение 90KB и 80KB:");
            var array90 = new byte[90 * 1024];
            var array80 = new byte[80 * 1024];
            Console.WriteLine($"byte[90KB] = {array90.Length} байт: поколение
{GC.GetGeneration(array90)} (LOH)");
            Console.WriteLine($"byte[80KB] = {array80.Length} байт: поколение
{GC.GetGeneration(array80)} (SOH)");
        }
    }

    public class LOHMonitor
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 5: МОНИТОРИНГ LOH ===\n");

            int[] sizesKB = { 10, 50, 90, 200 };
            var objects = new List<byte[]>();

            foreach (int size in sizesKB)
            {
                objects.Add(new byte[size * 1024]);
            }

            PrintGenerations(objects, "Сразу после создания");

            GC.Collect();
            PrintGenerations(objects, "После 1-го GC.Collect()");

            GC.Collect();
            PrintGenerations(objects, "После 2-го GC.Collect()");

            Console.WriteLine($"\nСборки: Gen0={GC.CollectionCount(0)},
Gen1={GC.CollectionCount(1)}, Gen2={GC.CollectionCount(2)}");
        }

        static void PrintGenerations(List<byte[]> objects, string message)
```

```
        {
            Console.WriteLine($"--- {message} ---");
            foreach (var obj in objects)
            {
                string heap = obj.Length > 85000 ? "LOH" : "SOH";
                Console.WriteLine($"{obj.Length / 1024,4} KB: поколение
{GC.GetGeneration(obj)} ({heap})");
            }
        }
    }
}

// ====================== Задание 6. Устранение упаковки ====================
namespace Lab2.Task6
{
    public class BoxingComparison
    {
        const int Count = 1000000;

        public static List<object> ProcessWithBoxing(out long sum)
        {
            var list = new List<object>();
            for (int i = 0; i < Count; i++)
            {
                list.Add(i); // упаковка
            }

            sum = 0;
            foreach (object item in list)
            {
                sum += (int)item; // распаковка
            }
            return list;
        }

        public static List<int> ProcessWithoutBoxing(out long sum)
        {
            var list = new List<int>();
            for (int i = 0; i < Count; i++)
            {
                list.Add(i);
            }

            sum = 0;
            foreach (int item in list)
            {
                sum += item;
            }
            return list;
        }

        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 6: СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ
===\n");

            ProcessWithBoxing(out _);      // прогрев JIT
            ProcessWithoutBoxing(out _);

            // память, которую занимают данные, измеряется через
GC.GetTotalMemory, пока список жив
            long memBefore = GC.GetTotalMemory(true);
```

```
            var sw = Stopwatch.StartNew();
            var boxedList = ProcessWithBoxing(out long boxedSum);
            sw.Stop();
            long boxedTime = sw.ElapsedMilliseconds;
            long boxedMemory = GC.GetTotalMemory(true) - memBefore;
            GC.KeepAlive(boxedList);
            boxedList = null;

            memBefore = GC.GetTotalMemory(true);
            sw.Restart();
            var unboxedList = ProcessWithoutBoxing(out long unboxedSum);
            sw.Stop();
            long unboxedTime = sw.ElapsedMilliseconds;
            long unboxedMemory = GC.GetTotalMemory(true) - memBefore;
            GC.KeepAlive(unboxedList);

            Console.WriteLine($"{"Метод",-22}|{"Время, мс",10} |{"Память,
КВ",12} |{"Сумма",14}");
            Console.WriteLine(new string('-', 64));
            Console.WriteLine($"{"ProcessWithBoxing",-22}|{boxedTime,10}
|{boxedMemory / 1024,12} |{boxedSum,14}");
            Console.WriteLine($"{"ProcessWithoutBoxing",-22}|{unboxedTime,10}
|{unboxedMemory / 1024,12} |{unboxedSum,14}");
            Console.WriteLine(new string('-', 64));
            Console.WriteLine($"Ускорение: {(double)boxedTime /
Math.Max(unboxedTime, 1):F1} раз, " +
                              $"экономия памяти: {(double)boxedMemory /
unboxedMemory:F1} раз");
        }
    }

    // Код из методички с упаковкой
    public class UnoptimizedCode
    {
        private ArrayList _data = new ArrayList();

        public void AddData(int value)
        {
            _data.Add(value); // Упаковка!
        }

        public int GetSum()
        {
            int sum = 0;
            foreach (object item in _data) // Распаковка!
            {
                sum += (int)item;
            }
            return sum;
        }
    }

    // Оптимизированная версия: List<int> хранит int без упаковки
    public class OptimizedCode
    {
        private List<int> _data = new List<int>();

        public void AddData(int value)
        {
            _data.Add(value);
        }
```

```
        public int GetSum()
        {
            int sum = 0;
            foreach (int item in _data)
            {
                sum += item;
            }
            return sum;
        }
    }

    public class OptimizationDemo
    {
        const int Count = 1000000;

        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 6: ОПТИМИЗАЦИЯ UnoptimizedCode
===\n");

            Measure("UnoptimizedCode (ArrayList)", () =>
            {
                var code = new UnoptimizedCode();
                for (int i = 0; i < Count; i++) code.AddData(i % 100);
                return code.GetSum();
            });

            Measure("OptimizedCode (List<int>)", () =>
            {
                var code = new OptimizedCode();
                for (int i = 0; i < Count; i++) code.AddData(i % 100);
                return code.GetSum();
            });
        }

        static void Measure(string name, Func<int> action)
        {
            action(); // прогрев JIT
            GC.Collect();

            long memBefore = GC.GetTotalAllocatedBytes(true);
            var sw = Stopwatch.StartNew();
            int sum = action();
            sw.Stop();
            long memory = GC.GetTotalAllocatedBytes(true) - memBefore;

            Console.WriteLine($"{name,-28}: время {sw.ElapsedMilliseconds,4}
мс, выделено {memory / 1024,6} КВ, сумма {sum}");
        }
    }
}

// ====================== Задание 7. Поиск утечек в коде ======================
namespace Lab2.Task7
{
    // Код с утечками из методички
    public class BuggyCode
    {
        private static List<IDisposable> _cache = new List<IDisposable>();
        private static event EventHandler _globalEvent;
        private byte[] _data = new byte[1024 * 1024];
```

```
        public BuggyCode()
        {
            _globalEvent += OnEvent;
            _cache.Add(new MemoryStream(1024));
        }

        private void OnEvent(object sender, EventArgs e)
        {
            var temp = new byte[1024 * 100];
        }

        public void SubscribeAll()
        {
            for (int i = 0; i < 1000; i++)
            {
                new BuggyCode();
            }
        }

        public static int CacheCount => _cache.Count;
    }

    // Исправленная версия
    public class FixedCode : IDisposable
    {
        private static event EventHandler _globalEvent;
        private MemoryStream _stream = new MemoryStream(1024); // ресурс
принадлежит объекту
        private byte[] _data = new byte[1024 * 1024];
        private bool _disposed;

        public FixedCode()
        {
            _globalEvent += OnEvent;
        }

        private void OnEvent(object sender, EventArgs e)
        {
            var temp = new byte[1024 * 100];
        }

        public void SubscribeAll()
        {
            for (int i = 0; i < 1000; i++)
            {
                using (var obj = new FixedCode())
                {
                    // работа с объектом
                }
            }
        }

        public void Dispose()
        {
            if (!_disposed)
            {
                _globalEvent -= OnEvent;  // отписка от статического события
                _stream.Dispose();        // освобождение IDisposable
                _stream = null;
                _data = null;
                _disposed = true;
            }
```

```
        }

        public static int SubscribersCount =>
_globalEvent?.GetInvocationList().Length ?? 0;
    }

    public class LeakSearch
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 7: ПОИСК УТЕЧЕК В КОДЕ ===\n");

            long before = GC.GetTotalMemory(true);
            using (var fixedCode = new FixedCode())
            {
                fixedCode.SubscribeAll();
            }
            long afterFixed = GC.GetTotalMemory(true);
            Console.WriteLine("FixedCode:");
            Console.WriteLine($" Память до: {before / (1024 * 1024)} MB,
после SubscribeAll и GC: {afterFixed / (1024 * 1024)} MB");
            Console.WriteLine($" Подписчиков статического события:
{FixedCode.SubscribersCount}");

            // SubscribeAll() у BuggyCode удерживает 1000 объектов по 1 MB
(около 1 GB),
            // поэтому для наглядного замера создаем 200 объектов тем же
способом
            before = GC.GetTotalMemory(true);
            for (int i = 0; i < 200; i++)
            {
                new BuggyCode();
            }
            long afterBuggy = GC.GetTotalMemory(true);
            Console.WriteLine("\nBuggyCode (200 объектов):");
            Console.WriteLine($" Память до: {before / (1024 * 1024)} MB,
после создания и GC: {afterBuggy / (1024 * 1024)} MB");
            Console.WriteLine($" Объектов MemoryStream в статическом кэше:
{BuggyCode.CacheCount}");
            Console.WriteLine($" Удержано памяти: {(afterBuggy - before) /
(1024 * 1024)} MB");
        }
    }
}
```